

ATmega32A Module

Timers (Normal, CTC and Fast PWM)

Ali Sahafi

1 Theory

A hardware timer is a counter that counts clock ticks *by itself*, independently of your program. The ATmega32A has three:

Timer	Width	Counts	Prescalers	OC pin(s)
Timer0	8-bit	0–255	1, 8, 64, 256, 1024	PB3 (LED D3)
Timer1	16-bit	0–65535	1, 8, 64, 256, 1024	PD5 (A), PD4 (B)
Timer2	8-bit	0–255	1, 8, 32, 64, 128, 256, 1024	PD7

Prescaler. The CPU clock is divided by the prescaler N before it reaches the counter, so one count takes $t_{\text{tick}} = N/F_{\text{CPU}}$. Example: at 8 MHz with $N = 1024$, one count lasts $128 \mu\text{s}$.

The three modes.

- **Normal** (TIMER_NORMAL): counts up to the maximum (255 or 65535), *overflows* back to 0 and sets the overflow flag TOV. To get a shorter delay, *preload* the counter:

$$t = (256 - \text{preload}) \cdot \frac{N}{F_{\text{CPU}}} \quad (65536 \text{ for Timer1})$$

- **CTC** (TIMER_CTC, *Clear Timer on Compare*): counts up to the compare value OCR, sets the compare flag OCF, restarts at 0. No preloading needed:

$$t = (\text{OCR} + 1) \cdot \frac{N}{F_{\text{CPU}}}$$

The timer can also *toggle* / *set* / *clear its OC pin* on every match, which produces a square wave with no CPU work at all.

- **Fast PWM** (TIMER_FAST_PWM): counts 0–255 over and over; the OC pin is switched at 0 and at the compare value, giving a PWM signal with a duty cycle set by OCR:

$$F_{\text{PWM}} = \frac{F_{\text{CPU}}}{N \cdot 256} \quad \text{Duty (active part)} = \frac{\text{OCR} + 1}{256}$$

2 API reference

Three ready-made objects exist: `Timer0`, `Timer1`, `Timer2`. Every timer is started with one call:

```
Timer0.begin(mode, prescaler);           // timer only
Timer0.begin(mode, prescaler, pinMode);  // timer + drive the OC pin
```

The options

Argument	Value	Meaning
mode	TIMER_NORMAL	Count 0 ... max, overflow, wrap to 0
	TIMER_CTC	Count 0 ... compare value, restart at 0
	TIMER_FAST_PWM	Count 0 ... 255, OC pin outputs PWM
prescaler	1, 8, 64, 256, 1024	Just write the divider you want. Timer2 also accepts 32 and 128.
	EXT_FALLING, EXT_RISING	Timer0/1 only: count pulses on pin T0 = PB0 / T1 = PB1 (counter mode)
pinMode	OC_OFF (default)	OC pin stays a normal GPIO pin
	<i>in TIMER_NORMAL / TIMER_CTC:</i>	
	OC_TOGGLE	Toggle the OC pin on every compare match
	OC_CLEAR	Drive the OC pin LOW on compare match
	OC_SET	Drive the OC pin HIGH on compare match
	<i>in TIMER_FAST_PWM:</i>	
	PWM_NON_INVERTING	HIGH from 0 until the compare value, then LOW
	PWM_INVERTING	LOW from 0 until the compare value, then HIGH. Use this for the active-low LEDs on the board.

When `pinMode` is not `OC_OFF`, the driver makes the OC pin an output automatically. While the timer drives it, writes to that pin through `GPIO` are ignored.

Methods

Method	Description
<code>Timer0.begin(mode, prescaler, pinMode)</code>	Configure and start (see options above)
<code>Timer0.setCount(value)</code>	Preload the counter (e.g. 156 for 100 counts in Normal mode)
<code>Timer0.getCount()</code>	Read the counter (0–255; Timer1: 0–65535)
<code>Timer0.setCompare(value)</code>	Compare value: CTC “count-to” value, or PWM level 0–255
<code>Timer0.setDutyCycle(percent)</code>	PWM only: 0–100 % of the period the pin is <i>active</i> (HIGH for <code>PWM_NON_INVERTING</code> , LOW for <code>PWM_INVERTING</code>). 0 % and 100 % are exact.
<code>Timer0.overflowed()</code>	<code>true</code> once per overflow — the flag is cleared for you
<code>Timer0.compareMatched()</code>	<code>true</code> once per compare match — the flag is cleared for you
<code>Timer0.stop()</code> / <code>Timer0.start()</code>	Pause / resume counting; settings are kept
<code>Timer0.onOverflow(function)</code>	<i>Optional:</i> call <code>function</code> on every overflow (interrupt)
<code>Timer0.onCompareMatch(function)</code>	<i>Optional:</i> call <code>function</code> on every compare match (interrupt)
<i>Timer1 only (two OC channels, A = PD5, B = PD4):</i>	
<code>Timer1.begin(mode, N, pinModeA, pinModeB)</code>	Channel B gets its own <code>pinMode</code>
<code>Timer1.setCompareA/B(value)</code>	16-bit compare values. <code>setCompare()</code> = channel A; CTC counts up to A
<code>Timer1.setDutyCycleA/B(percent)</code>	PWM duty per channel. <code>setDutyCycle()</code> = channel A
<code>Timer1.compareMatchedA/B()</code>	Compare flags per channel. <code>compareMatched()</code> = channel A
<code>Timer1.setTop(top)</code>	PWM period in counts (default 255): $T_{\text{PWM}} = (\text{top} + 1) \cdot N / F_{\text{CPU}}$

Waiting for a flag (polling). `overflowed()` and `compareMatched()` behave like “has it happened since I last asked?”. So `while (!Timer0.overflowed());` simply waits for the next overflow. You never write 1 to TIFR yourself.

Interrupts (optional). `onOverflow()` and `onCompareMatch()` enable the interrupt and `sei()` for you. The function runs in the background; keep it short and declare variables it shares with `main()` as `volatile`. Pass `nullptr` to switch it off.

What the driver does in the registers

The lecture slides configure the registers by hand. Each driver call is exactly one of those steps:

Driver call	Registers (Timer0)	Bits
Timer0.begin(TIMER_NORMAL, 1024)	TCCR0 = 0x05;	CS02 CS00
Timer0.begin(TIMER_CTC, 1024, OC_TOGGLE)	TCCR0 = 0x1D;	WGM01 COM00 CS02 CS00
	DDRB = 1<<PB3;	OC0 pin as output
Timer0.begin(TIMER_FAST_PWM, 1, PWM_INVERTING)	TCCR0 = 0x79;	WGM00 WGM01 COM01 COM00 CS00
Timer0.setCount(156)	TCNT0 = 156;	
Timer0.setCompare(x)	OCR0 = x;	
Timer0.overflowed()	TIFR & (1<<TOV0)	then TIFR = 1<<TOV0;
Timer0.compareMatched()	TIFR & (1<<OCF0)	then TIFR = 1<<OCF0;

Warning: F_CPU and the fuses. All timing formulas use the *real* clock. `make` (and `make timer`) always programs the fuses for the external 8 MHz crystal. To run at another clock, first set the fuses (e.g. `make int-1mhz`), then flash *without* touching the fuses: `make build flash SRC=Task.cpp`. Also change `#define F_CPU` so `_delay_ms()` stays correct.

3 Example: blink + PWM, all in hardware

Board Hardware. All components are already on the development board:

- **8 LEDs (D0–D7) on PORTB**, active-low (LOW = ON). LED **D3** sits on PB3 = OC0, the Timer0 output pin.
- **Seven-segment display** also on PORTB (shares the pins with the LEDs).
- **8 DIP switches (S0–S7) on PORTC**, closed = LOW; use INPUT_PULLUP.
- **Push buttons S11 on PD2 and S10 on PD6**, pressed = LOW.

Build & flash. make timer

```
/*
 * Timer Example -- one timer per mode, all running at the same time
 * Build & flash: make timer
 *
 * LED D0 (PB0): blinks every 0.5 s -- Timer1, CTC mode, flag polling
 * LED D3 (PB3): half brightness -- Timer0, Fast PWM on its OC0 pin
 *
 * Timer1 CTC period = prescaler * (compare + 1) / F_CPU
 *                   = 256 * (15624 + 1) / 8 MHz = 0.5 s
 */

#define F_CPU 8000000UL
#include "../gpio/gpio.hpp"
#include "timer.hpp"

int main() {
    GPIO.setDirection(PORTB, PB0, OUTPUT); // LED D0 (active-low)

    // Timer0: Fast PWM, prescaler 64 -> 8 MHz / (64 * 256) = 488 Hz, no flicker.
    // PWM_INVERTING because the LEDs are active-low (duty cycle = brightness).
    Timer0.begin(TIMER_FAST_PWM, 64, PWM_INVERTING); // PB3 becomes an output
    Timer0.setDutyCycle(50);

    // Timer1: CTC, restarts every 15625 counts = 0.5 s
    Timer1.begin(TIMER_CTC, 256);
    Timer1.setCompare(15624);

    while (true) {
        if (Timer1.compareMatched()) { // true once every 0.5 s
            GPIO.toggle(PORTB, PB0);
        }
        // The CPU is free here -- the PWM on D3 needs no code at all.
    }
}
```

4 Laboratory Tasks

Create one `.cpp` file per task in the project root (e.g. `Task1.cpp`), start it with `#define F_CPU` and include the drivers you need (`drivers/gpio/gpio.hpp`, `drivers/timer/timer.hpp`, `drivers/common/sevenseg.hpp`), then flash with `make SRC=Task1.cpp`.

Task 0: Fuse Bits and F_CPU (10 min)

Count 0–9 on the seven-segment display, one step per second, using `SevenSeg(digit)` and `_delay_ms(1000)`.

- **Task 0.1 (Internal 1 MHz):** Run `make int-1mhz`, set `#define F_CPU 1000000UL` and flash with `make build flash SRC=Task0.cpp`. Check the timing with a stopwatch.
- **Task 0.2 (External 8 MHz crystal):** Set `#define F_CPU 8000000UL` and flash with `make SRC=Task0.cpp` (this also selects the crystal).
- **Task 0.3 (Mismatch):** Keep `F_CPU 8000000UL` but switch the chip back to `make int-1mhz`. How long is one “second” now, and why?

Task 1: Timer Normal Mode — Counter on the LEDs (30 min)

Count in binary on the 8 LEDs, one step every **100 ms**, timed by `Timer0` in `TIMER_NORMAL` mode (external 8 MHz crystal). The LEDs are active-low, so write the inverted count: `GPIO.write(PORTB, ~count)`.

- **Task 1.1 (Calculate first):** With prescaler 1024, how long is one count, and what is the *longest* time a `Timer0` overflow can take? Why can a single overflow not give 100 ms?
- **Task 1.2 (10 × 10 ms):** Compute the preload value for a 10 ms overflow. Then, in the main loop, repeat 10 times: preload with `Timer0.setCount()` and wait with `while (!Timer0.overflowed());`. Then increment the counter.
- **Task 1.3 (16-bit Timer1):** Do the same with `Timer1`, which can reach 100 ms in *one* overflow. Which prescalers make the preload value an exact integer? Compare the code with Task 1.2.

Task 2: Timer CTC Mode — Hardware Blinker (20 min)

Let the timer blink LED **D3** (`PB3 = 0C0`) by itself, with the blink speed set by the DIP switches on `PORTC`.

- **Task 2.1 (Blink in hardware):** Start `Timer0` in `TIMER_CTC` mode, prescaler 1024, with `OC_TOGGLE`. The main loop only copies the switch value (`GPIO.read(PINC)`) into the compare value with `Timer0.setCompare()` — no flag checking needed. At 8 MHz, compute the slowest possible blink frequency. What do you see on D3, and why?
- **Task 2.2 (Make it visible):** Switch the chip to the internal 1 MHz clock (see Task 0.1). What is the toggle interval range now? Verify with the switches.
- **Task 2.3 (It runs in parallel):** Keep the blinker and, in the same loop, also show the switch value on the other LEDs: `GPIO.write(PORTB, GPIO.read(PINC))`. Why does D3 keep blinking although the loop also writes to PB3?

Task 3: Timer Fast PWM — LED Brightness (25 min)

Control the brightness of LED **D3** (`0C0`) with PWM on `Timer0`.

- **Task 3.1 (Switches → brightness):** Start `Timer0` in `TIMER_FAST_PWM` with `PWM_INVERTING` (the LED is active-low). Choose the prescaler so that $F_{\text{PWM}} \approx 4 \text{ kHz}$ at 8 MHz. In the loop: `Timer0.setCompare(GPIO.read(PINC))`.

- **Task 3.2 (Buttons in percent):** Replace the switches with the push buttons: S11 (PD2) raises the brightness by 10 %, S10 (PD6) lowers it by 10 %, limited to 0–100 %. Use `Timer0.setDutyCycle(percent)`. Act once per press (wait for release).
- **Task 3.3 (Flicker):** Compute F_{PWM} for prescalers 64, 256 and 1024. Try them: from which one on can you see the LED flicker?
- **Task 3.4 (Breathing LED, two timers):** Fade D3 smoothly from 0 to 100 % and back, one 1 % step every 10 ms. Instead of `_delay_ms()`, use `Timer1` in `TIMER_CTC` mode as a 10 ms time base (`Timer1.compareMatched()`). Compute its prescaler and compare value.